Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

PY102

Challenge: identifying Big O

Today
Welcome to your next lesson: Challenge: identifying Big O.

I'm getting your personalized lesson ready, based on what you've learned, what's next, and how you learn best.

Step by step, you're moving forward.

Let's jump into some Big O practice, one small piece at a time

We'll look at tiny Python snippets and you'll tell me how their running time grows: O(1), O(n), or O(n²).

First, quick check so I can tune the difficulty:

Have you already practiced counting loop iterations and labeling code as O(1) / O(n) / O(n²), or does it still feel shaky?

You can just answer: "comfortable" or "shaky".

Comfortable

What do you think this one is?

Round 1

You have a list of n users, and you just want to print the first user:

```
users = ["a", "b", "c", ...]  # length n
print(users[0])
```

What is the time complexity in terms of n?

A.

O(1)

B.

O(n)

C.

O(n²)

Nice work picking O(1) on that first one 🎯

Accessing users[0] is a single, constant-time step — it doesn't matter if the list has 10 users or 10 million, it's still just "grab the first slot and print it", so that's O(1).

Now, take your time with the next question below 👇

Round 2 ⚡

You have a list of n messages and you want to print all of them once:

```
messages = ["m1", "m2", "m3", ...]  # length n
for msg in messages:
    print(msg)
```

What is the time complexity in terms of n (worst case)?

A.

O(1)

B.

O(n)

C.

O(n^2)

You're on a roll, Ron 🎯

Looping once over all n messages is O(n) because:

The work grows linearly with the number of items.

Double the messages → about double the print calls → linear growth.

Ready for the next round?

(If yes, just say "go" or "next" and I'll throw you the next snippet.)

Go to the next one

What do you think for this one? Take your time with the question below 👇

Round 3 🔁

You have a list of n numbers and you want to print every possible pair (a, b) from the list (including pairs where a and b are the same element):

```
nums = [1, 2, 3, ...]  # length n
for a in nums:
    for b in nums:
        print(a, b)
```

What is the time complexity in terms of n (worst case)?

A.

O(1)

B.

O(n)

C.

O(n²)

Love it — you spotted the O(n²) correctly ✨

Reason step by step:

Outer loop runs n times (for each a).
For each a, the inner loop runs n times (for each b).
Total prints ≈ n * n = n² → that's quadratic time, O(n²).
Ready for another one, or want a quick pause to recap the patterns so far (O(1) vs O(n) vs O(n²))?

Next one

What do you think? Take your time with the question above 👆

Round 4 🔍 (best vs worst case)

You search for a name in a list of n contacts and stop as soon as you find a match:

```
contacts = ["Alice", "Bob", "Cara", ...]  # length n
name = "Zoe"

for c in contacts:
    if c == name:
        print("Found!")
        break
```

In Big O (worst case), what is the time complexity in terms of n?

A.

O(1)

B.

O(n)

C.

O(n²)

Good attempt here — this one is sneaky 😉

In the worst case, the name isn't in the list at all (or it's the last item). Then the loop checks every contact once, so the work grows linearly with n → O(n).

Key idea:

Best case: the first contact matches → runs 1 time → looks like O(1).
Worst case (what Big O asks for here): no match or last contact → runs n times → O(n).
Want to try another one to lock in this best vs worst-case idea?

Give me another example

What do you think the answer is here? 👆 Take your time and pick one.

Round 5 📄

You have a list of n temperatures and you want to:

```
Print the first temperature
Then print all temperatures
temps = [70, 71, 69, ...]  # length n
print(temps[0])

for t in temps:
    print(t)
What is the overall time complexity in terms of n (worst case)?

A.

O(1)

B.

O(n)

C.

O(n²)

Nice, Ron — nailed that one 🎯


Step-by-step:


print(temps[0]) is a single constant-time step → O(1).
The for loop goes through all n temps once → O(n).
Total: O(1) + O(n) → we ignore the constant and lower-order term → overall O(n).
You asked for another example earlier — want to do one more challenge to practice best vs worst case again, or are you feeling ready to move on from this lesson’s Big O practice?

One more challenge

What do you think? Take your time with the question above 👆

Final Round 6 🎯 (compare two solutions)

You have a list of n items and you want to check if any of them is equal to 0.

Version A:

items = [1, 5, 0, 3, ...]  # length n
found = False

for x in items:
    if x == 0:
        found = True
        break
Version B:

items = [1, 5, 0, 3, ...]  # length n
found = False

for x in items:
    if x == 0:
        found = True

print(found)
In Big O (worst case), which statement is true?

A.

Both Version A and Version B are O(n) in the worst case

B.
```

Version A is O(1) and Version B is O(n) in the worst case

C.

Version A is O(n) and Version B is O(n²) in the worst case

Beautifully done on that last one, Ron 🎯

Your answer is spot on:

In the worst case, both Version A and Version B may have to look at all n items.
Version A can stop early with break (great for best case), but if 0 is missing or at the end, it still checks every item → O(n).
Version B always scans all items and sets found when needed → also O(n).
So: both are O(n) in worst case — Version A just has a better best case.
You've just:

Correctly identified O(1), O(n), O(n²) from code.
Counted loop behavior (single loop vs nested loops vs constant work).
Compared two correct solutions using worst-case Big O, and noticed how early exit affects best case but not worst case.
That's exactly what this challenge lesson was about ☑

From here, you can:

Move on to the next lesson in Data Structures & Algorithms, or
If you want a little extra practice, I can invent a few more real-world Big O puzzles just for you.
What do you feel like: next lesson or extra practice?